

Problemas Soma de dois e Soma de Subconjunto

Prof. Thiago Braga Marcilon

Universidade Federal do Cariri

CC0095 – Tópicos Avançados em Complexidade

Problema Soma de Dois

Soma de dois (Two sum)

Problema Soma de Dois

Soma de dois (Two sum)

- **Entrada:** um array A com n inteiros e um inteiro k ;

Problema Soma de Dois

Soma de dois (Two sum)

- **Entrada:** um array A com n inteiros e um inteiro k ;
- **Tarefa:** determinar se existem duas posições distintas de A cuja soma dos elementos seja k ;

Duas ideias

Duas ideias

- 1 Ordena A e trabalha com A ordenado;

Duas ideias

- 1 Ordena A e trabalha com A ordenado;
 - Truque dos dois índices;

Duas ideias

- 1 Ordena A e trabalha com A ordenado;
 - Truque dos dois índices;
 - Se A já vier ordenado, melhor ainda!

Duas ideias

- 1 Ordena A e trabalha com A ordenado;
 - Truque dos dois índices;
 - Se A já vier ordenado, melhor ainda!
- 2 Trabalha com A desordenado;

Duas ideias

- 1 Ordena A e trabalha com A ordenado;
 - Truque dos dois índices;
 - Se A já vier ordenado, melhor ainda!
- 2 Trabalha com A desordenado;
 - Usar tabela hash!

Prós e Contras

Prós

Prós e Contras

Prós

- Ordenando A :

Prós e Contras

Prós

- Ordenando A :
 - Requer $O(1)$ de espaço;

Prós

- Ordenando A :
 - Requer $O(1)$ de espaço;
 - Não requer uso de estrutura de dados “complexa”

Prós

- Ordenando A :
 - Requer $O(1)$ de espaço;
 - Não requer uso de estrutura de dados “complexa”
- Table hash:

Prós

- Ordenando A :
 - Requer $O(1)$ de espaço;
 - Não requer uso de estrutura de dados “complexa”
- Table hash:
 - Tempo esperado é $O(n)$;

Prós

- Ordenando A :
 - Requer $O(1)$ de espaço;
 - Não requer uso de estrutura de dados “complexa”
- Table hash:
 - Tempo esperado é $O(n)$;
 - Se a tabela hash já estiver implementada nativamente, o código é limpo.

Prós e Contras

Contras

Contras

- Ordenando A :

Contras

- Ordenando A :
 - Pior caso $O(n \log n)$ se A vier desordenado;

Contras

- Ordenando A :
 - Pior caso $O(n \log n)$ se A vier desordenado;
- Table hash:

Contras

- Ordenando A :
 - Pior caso $O(n \log n)$ se A vier desordenado;
- Table hash:
 - Se der azar, fica $O(n^2)$;

Contras

- Ordenando A :
 - Pior caso $O(n \log n)$ se A vier desordenado;
- Table hash:
 - Se der azar, fica $O(n^2)$;
 - Se uma implementação não estiver disponível, é necessário implementar a tabela hash.

Problema Soma de Subconjunto

Soma de Subconjunto

Problema Soma de Subconjunto

Soma de Subconjunto

- **Entrada:** um array A com n natural e um natural k ;

Problema Soma de Subconjunto

Soma de Subconjunto

- **Entrada:** um array A com n natural e um natural k ;
- **Tarefa:** determinar se existe um subconjunto de posições de A cuja soma dos elementos seja k ;

Duas ideias

Duas ideias

- 1 Programação dinâmica:

Duas ideias

1 Programação dinâmica:

- para cada $0 \leq i < n$ e $0 \leq j \leq k$, determinar $L[i, j]$, ou seja, se existe um subconjunto de posições de A entre 0 e i cuja soma dos elementos seja j ;

Duas ideias

1 Programação dinâmica:

- para cada $0 \leq i < n$ e $0 \leq j \leq k$, determinar $L[i, j]$, ou seja, se existe um subconjunto de posições de A entre 0 e i cuja soma dos elementos seja j ;
- $L[i, j] = L[i - 1, j]$ **ou** $L[i - 1, j - A[i]]$

Duas ideias

1 Programação dinâmica:

- para cada $0 \leq i < n$ e $0 \leq j \leq k$, determinar $L[i, j]$, ou seja, se existe um subconjunto de posições de A entre 0 e i cuja soma dos elementos seja j ;
- $L[i, j] = L[i - 1, j]$ **ou** $L[i - 1, j - A[i]]$

2 Meet in the middle:

Duas ideias

1 Programação dinâmica:

- para cada $0 \leq i < n$ e $0 \leq j \leq k$, determinar $L[i, j]$, ou seja, se existe um subconjunto de posições de A entre 0 e i cuja soma dos elementos seja j ;
- $L[i, j] = L[i - 1, j]$ **ou** $L[i - 1, j - A[i]]$

2 Meet in the middle:

- Calcula por força bruta todas as somas de todos os subconjunto de cada metade de A separadamente;

Duas ideias

1 Programação dinâmica:

- para cada $0 \leq i < n$ e $0 \leq j \leq k$, determinar $L[i, j]$, ou seja, se existe um subconjunto de posições de A entre 0 e i cuja soma dos elementos seja j ;
- $L[i, j] = L[i - 1, j]$ **ou** $L[i - 1, j - A[i]]$

2 Meet in the middle:

- Calcula por força bruta todas as somas de todos os subconjunto de cada metade de A separadamente;
- Ordena a soma das duas metades e aplica a mesma ideia to two sum.

Prós e Contras

Prós

Prós e Contras

Prós

- Programação dinâmica:

Prós

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: rápido e econômico quando k é pequeno!

Prós

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: rápido e econômico quando k é pequeno!
 - Fácil implementação;

Prós

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: rápido e econômico quando k é pequeno!
 - Fácil implementação;
- Meet in the middle:

Prós

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: rápido e econômico quando k é pequeno!
 - Fácil implementação;
- Meet in the middle:
 - Tempo e espaço dependem apenas de n não importando o valor de k ($O(2^{n/2})$);

Prós

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: rápido e econômico quando k é pequeno!
 - Fácil implementação;
- Meet in the middle:
 - Tempo e espaço dependem apenas de n não importando o valor de k ($O(2^{n/2})$);
 - Facilmente adaptável em problemas de outros domínios.

Prós e Contras

Contras

Contras

- Programação dinâmica:

Contras

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: inviável quando k é grande, pois é exponencial no tamanho de k ($\log k$);

Contras

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: inviável quando k é grande, pois é exponencial no tamanho de k ($\log k$);
 - Estratégia específica aplicável apenas para variações próximas;

Contras

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: inviável quando k é grande, pois é exponencial no tamanho de k ($\log k$);
 - Estratégia específica aplicável apenas para variações próximas;
- Meet in the middle:

Contras

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: inviável quando k é grande, pois é exponencial no tamanho de k ($\log k$);
 - Estratégia específica aplicável apenas para variações próximas;
- Meet in the middle:
 - Tempo exponencial em n ;

Contras

- Programação dinâmica:
 - Tempo e espaço pseudopolinomial $O(nk)$: inviável quando k é grande, pois é exponencial no tamanho de k ($\log k$);
 - Estratégia específica aplicável apenas para variações próximas;
- Meet in the middle:
 - Tempo exponencial em n ;
 - Implementação mais complicada.